

EXPERIMENT – 9

Name: K ANUSRI

Roll Number: 2520030432

Section: 8

Title

Directory Navigation and Built-in Command Dispatch

Aim

To change directories, validate paths, update the working directory, handle directory errors, maintain the previous directory, and test navigation. To identify built-in commands, create a dispatch table, execute in-process commands, handle invalid commands, maintain state, and test dispatch logic.

Theory

Directory navigation in Linux can be performed using the `chdir()` system call. The `getcwd()` function is used to obtain the current working directory. Before changing directories, a path can be validated using `stat()` and checked to determine whether it refers to a directory.

The program maintains both the current and previous working directories. When a directory change is successful, the old working directory is saved as the previous directory. This allows the program to return to the previous location when required.

Built-in commands are handled directly by the program. A dispatch table maps command names to their corresponding functions. Commands such as `cd`, `pwd`, `prev`, `help`, and `test` can therefore be executed in-process while the program maintains its navigation state. Invalid commands are reported when they are not present in the dispatch table.

Algorithm

1. Start the program.
2. Obtain and store the initial working directory using `getcwd()`.
3. Create variables to maintain the current and previous directories.
4. Create a dispatch table containing the supported built-in commands.
5. Validate a requested directory using `stat()` and check whether it is a directory.
6. Change the working directory using `chdir()`.
7. Store the old directory as the previous directory and update the current directory.
8. Provide a previous-directory operation to return to the stored location.
9. Handle invalid directory paths and display an appropriate error.
10. Execute built-in commands directly using the dispatch table.
11. Test directory navigation using the parent directory and previous-directory operation.
12. Handle commands that are not present in the dispatch table.
13. Display the maintained directory state.
14. Stop the program.

CODE

```
k_anusri@DESKTOP-TM20BA( X + v
k_anusri@DESKTOP-TM20BAQ:~$ #include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/stat.h>

#define MAX_LINE 512
#define MAX_ARGS 20

char current_directory[PATH_MAX];
char previous_directory[PATH_MAX];

void show_current_directory()
{
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current Directory: %s\n", cwd);
    else
        perror("getcwd");
}

int validate_directory(const char *path)
{
    struct stat info;

    if (stat(path, &info) != 0)
        return 0;

    return S_ISDIR(info.st_mode);
}

int change_directory(const char *path)
{
    char old_directory[PATH_MAX];
    char new_directory[PATH_MAX];

    if (getcwd(old_directory, sizeof(old_directory)) == NULL)
    {
        perror("getcwd");
        return 0;
    }

    if (!validate_directory(path))
    {
        printf("Error: Invalid directory path: %s\n", path);
        return 0;
    }

    if (chdir(path) != 0)
    {
        perror("chdir");
        return 0;
    }

    if (getcwd(new_directory, sizeof(new_directory)) == NULL)
    {
        perror("getcwd");
        return 0;
    }

    strcpy(previous_directory, old_directory);
    strcpy(current_directory, new_directory);
}
```



k_anusri@DESKTOP-TM20BAC



```
    printf("Directory changed successfully.\n");
    printf("Previous Directory: %s\n", previous_directory);
    printf("Current Directory : %s\n", current_directory);

    return 1;
}

void go_previous_directory()
{
    char temp[PATH_MAX];

    if (strlen(previous_directory) == 0)
    {
        printf("Previous directory is not available.\n");
        return;
    }

    strcpy(temp, current_directory);

    if (chdir(previous_directory) != 0)
    {
        perror("chdir");
        return;
    }

    strcpy(current_directory, previous_directory);
    strcpy(previous_directory, temp);

    printf("Returned to previous directory.\n");
    printf("Current Directory : %s\n", current_directory);
    printf("Previous Directory: %s\n", previous_directory);
}

void builtin_pwd(char *args[], int count)
{
    show_current_directory();
}

void builtin_cd(char *args[], int count)
{
    if (count < 2)
    {
        printf("Usage: cd <directory>\n");
        return;
    }

    change_directory(args[1]);
}

void builtin_prev(char *args[], int count)
{
    go_previous_directory();
}

void builtin_help(char *args[], int count)
{
    printf("\nBuilt-in Commands\n");
    printf("-----\n");
    printf("cd <path>    - Change directory\n");
    printf("pwd          - Display current directory\n");
    printf("prev         - Return to previous directory\n");
    printf("help        - Display built-in commands\n");
    printf("test        - Test directory navigation\n");
    printf("exit        - Exit the program\n");
}
```



k_anusri@DESKTOP-TM20BAC



```
}

void builtin_test(char *args[], int count)
{
    char start[PATH_MAX];

    printf("\nNavigation Test\n");
    printf("-----\n");

    if (getcwd(start, sizeof(start)) == NULL)
    {
        perror("getcwd");
        return;
    }

    printf("Starting Directory: %s\n", start);

    printf("\nTesting current directory...\n");
    show_current_directory();

    printf("\nTesting parent directory using '..'...\n");

    if (change_directory(".."))
    {
        printf("Parent directory reached successfully.\n");

        printf("\nReturning to previous directory...\n");
        go_previous_directory();
    }

    printf("\nNavigation test completed.\n");
}

typedef struct
{
    char *name;
    void (*function)(char *[], int);
} Builtin;

Builtin dispatch_table[] =
{
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"prev", builtin_prev},
    {"help", builtin_help},
    {"test", builtin_test}
};

int dispatch_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);

int execute_builtin(char *args[], int count)
{
    int i;

    for (i = 0; i < dispatch_count; i++)
    {
        if (strcmp(args[0], dispatch_table[i].name) == 0)
        {
            dispatch_table[i].function(args, count);
            return 1;
        }
    }
}
```



k_anusri@DESKTOP-TM20BA



```
    }

    if (strcmp(args[0], "exit") == 0)
    {
        printf("Exiting directory shell...\n");
        exit(0);
    }

    return 0;
}

int main()
{
    char line[MAX_LINE];
    char *args[MAX_ARGS];
    char *token;
    int count;
    int i;

    if (getcwd(current_directory, sizeof(current_directory)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    previous_directory[0] = '\\0';

    printf("=====\n");
    printf(" DIRECTORY NAVIGATION AND BUILT-IN DISPATCH\n");
    printf("=====\n");

    printf("\n1. INITIAL WORKING DIRECTORY\n");
    printf("-----\n");
    printf("Current Directory: %s\n", current_directory);

    printf("\n2. BUILT-IN COMMAND DISPATCH TABLE\n");
    printf("-----\n");
    printf("Registered built-ins:\n");

    for (i = 0; i < dispatch_count; i++)
        printf("%s\n", dispatch_table[i].name);

    printf("exit\n");

    printf("\n3. DIRECTORY NAVIGATION TEST\n");
    printf("-----\n");

    {
        char *test1[] = {"pwd"};
        execute_builtin(test1, 1);
    }

    {
        char *test2[] = {"cd", ".."};
        execute_builtin(test2, 2);
    }

    printf("\nReturning to previous directory...\n");

    {
        char *test3[] = {"prev"};
        execute_builtin(test3, 1);
    }
}
```

```
}

printf("\n4. INVALID PATH TEST\n");
printf("-----\n");

{
    char *test4[] = {"cd", "/invalid/path/xyz123"};
    execute_builtin(test4, 2);
}

printf("\n5. INTERACTIVE COMMAND TEST\n");
printf("-----\n");
printf("Enter commands below.\n");
printf("Examples: pwd, cd .., prev, test, help\n");
printf("Type exit to stop.\n\n");

while (1)
{
    printf("directory-shell> ");

    if (fgets(line, sizeof(line), stdin) == NULL)
        break;

    line[strcspn(line, "\n")] = '\0';

    if (strlen(line) == 0)
        continue;

    count = 0;
    token = strtok(line, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count++] = token;
        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (count == 0)
        continue;

    if (!execute_builtin(args, count))
    {
        printf("Invalid command: %s\n", args[0]);
        printf("Command is not present in dispatch table.\n");
    }
}

return 0;
}
```

Execution

1. Directory navigation, built-in dispatch table, previous-directory handling, and invalid path test

```
=====
  DIRECTORY NAVIGATION AND BUILT-IN DISPATCH
=====

1. INITIAL WORKING DIRECTORY
-----
Current Directory: /home/mohith_07

2. BUILT-IN COMMAND DISPATCH TABLE
-----
Registered built-ins:
cd
pwd
prev
help
test
exit

3. DIRECTORY NAVIGATION TEST
-----
Current Directory: /home/mohith_07
Directory changed successfully.
Previous Directory: /home/mohith_07
Current Directory : /home

Returning to previous directory...
Returned to previous directory.
Current Directory : /home/mohith_07
Previous Directory: /home

4. INVALID PATH TEST
-----
Error: Invalid directory path: /invalid/path/xyz123

5. INTERACTIVE COMMAND TEST
-----
Enter commands below.
Examples: pwd, cd .., prev, test, help
Type exit to stop.
```

Result

The program was executed successfully. The current working directory was identified and directory navigation was tested using the cd operation. Directory paths were validated before changing the working directory, the previous directory was maintained, and invalid paths were handled with an error message. Built-in commands were registered through a dispatch table and prepared for in-process execution.